RAPPORT D'ATELIER

Atelier 1

Gestion Commerciale

Base de données MySQL • SQL • MySQL Workbench

Réalisé par

Basma Elmaimouni

Module M203

10 novembre 2026

Sommaire

1	Conception de la base
2	Requêtes de jointures
3	Agrégations et calculs
4	Fonction SQL
5	IN / NOT IN / EXISTS
6	Optimisation : Index et Vues
7	DCL - Gestion des utilisateurs

Environnement

Les requêtes ont été exécutées avec MySQL Workbench sur une instance locale (WampServer), dans la base de données atelier_commerce.

1 Conception de la base

Objectif Création de la base de données atelier_commerce, des cinq tables (clients, produits, commandes, details_commandes, paiements) avec leurs clés primaires et étrangères, puis insertion des données de test.

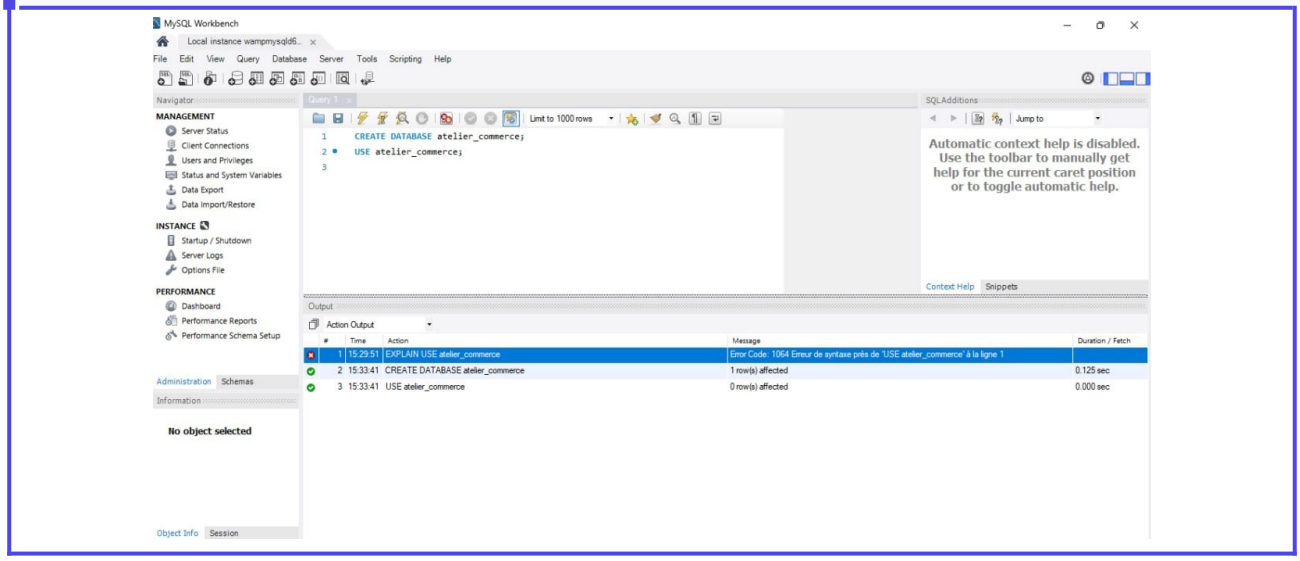


Figure 1.1 — Création de la base atelier_commerce et sélection avec USE.

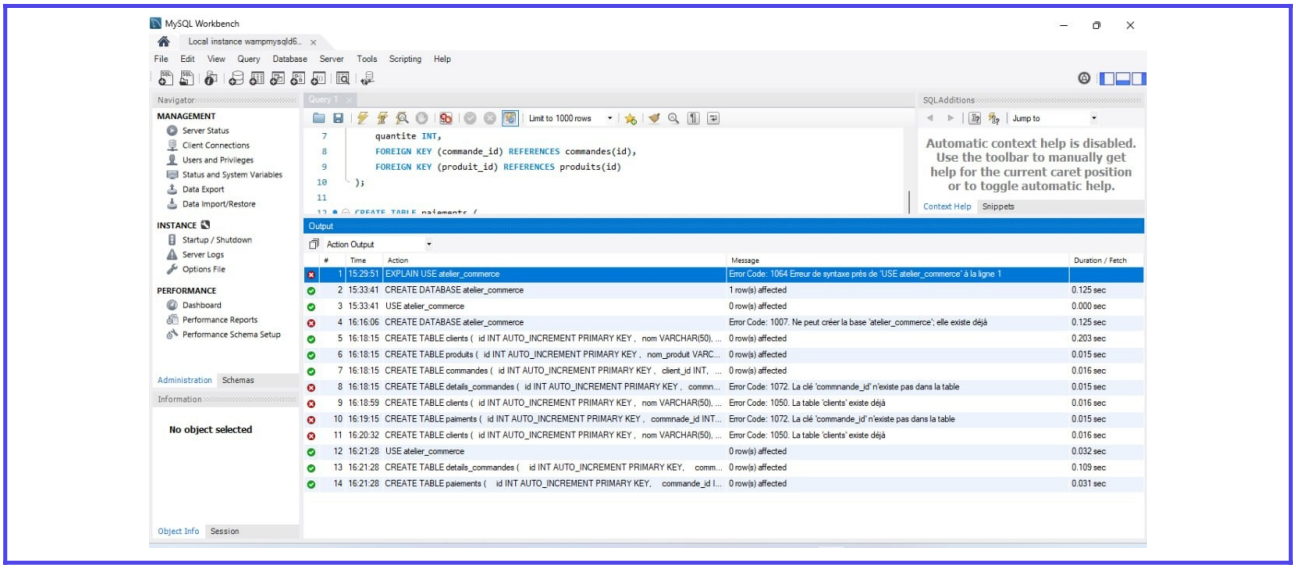


Figure 1.2 — Création des tables : l'historique des exécutions montre les erreurs corrigées puis les tables créées avec succès.

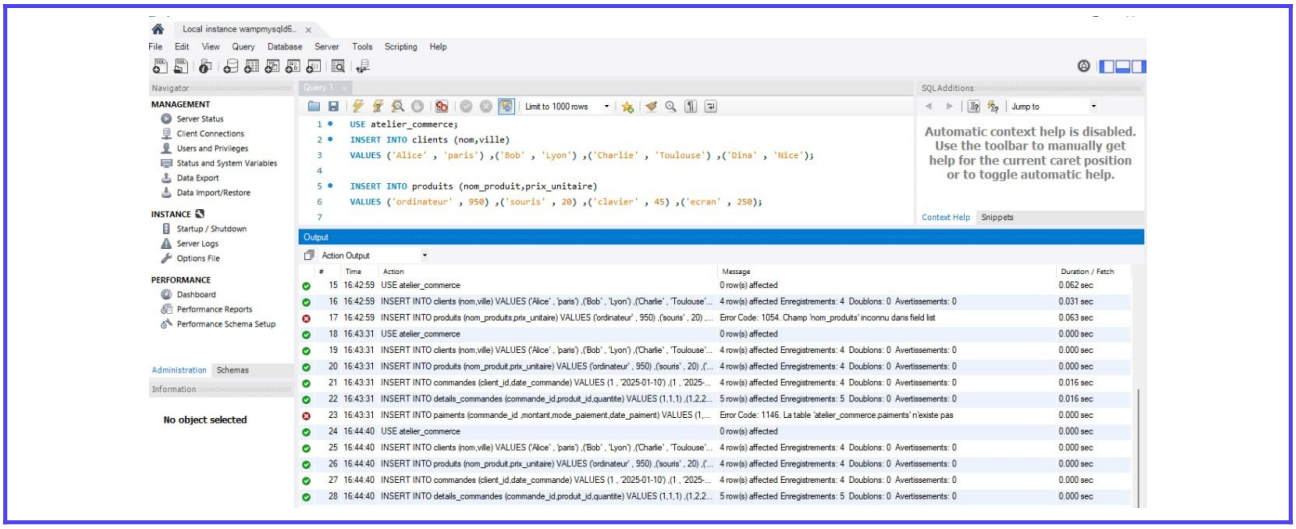


Figure 1.3 — *Insertion des données dans clients, produits, commandes et details_commandes.*

2 Requêtes de jointures

Objectif Utilisation des jointures INNER JOIN, LEFT JOIN, RIGHT JOIN et de la simulation de FULL JOIN (LEFT JOIN + UNION + RIGHT JOIN) pour relier clients, commandes, détails et produits.

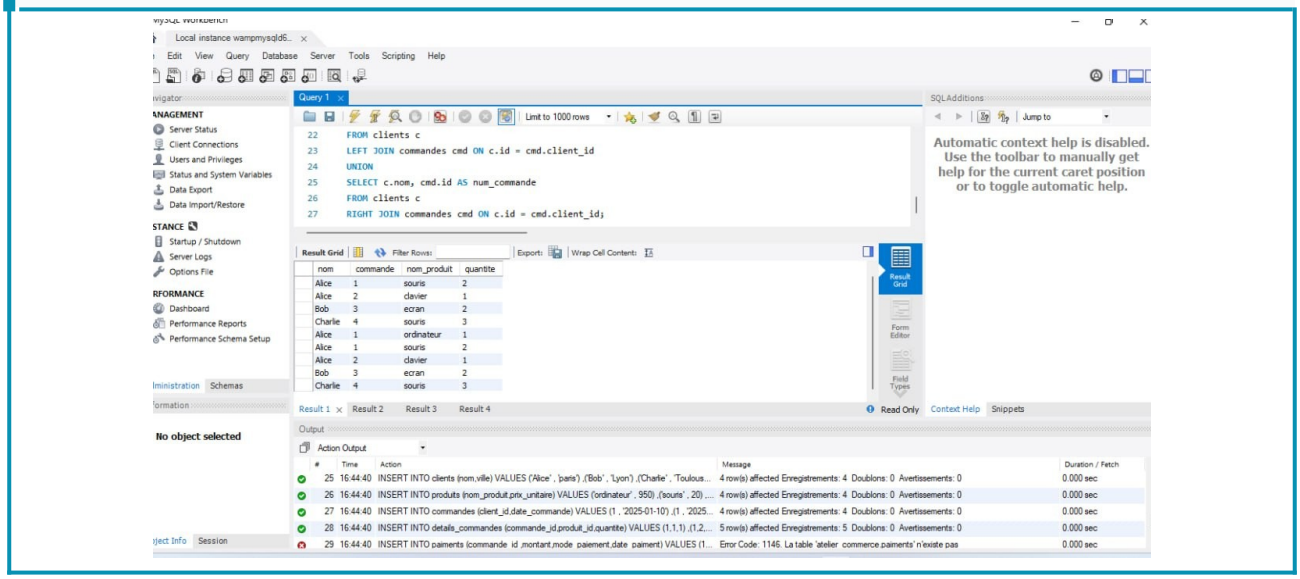


Figure 2.1 — INNER JOIN : clients, commandes, produits et quantités.

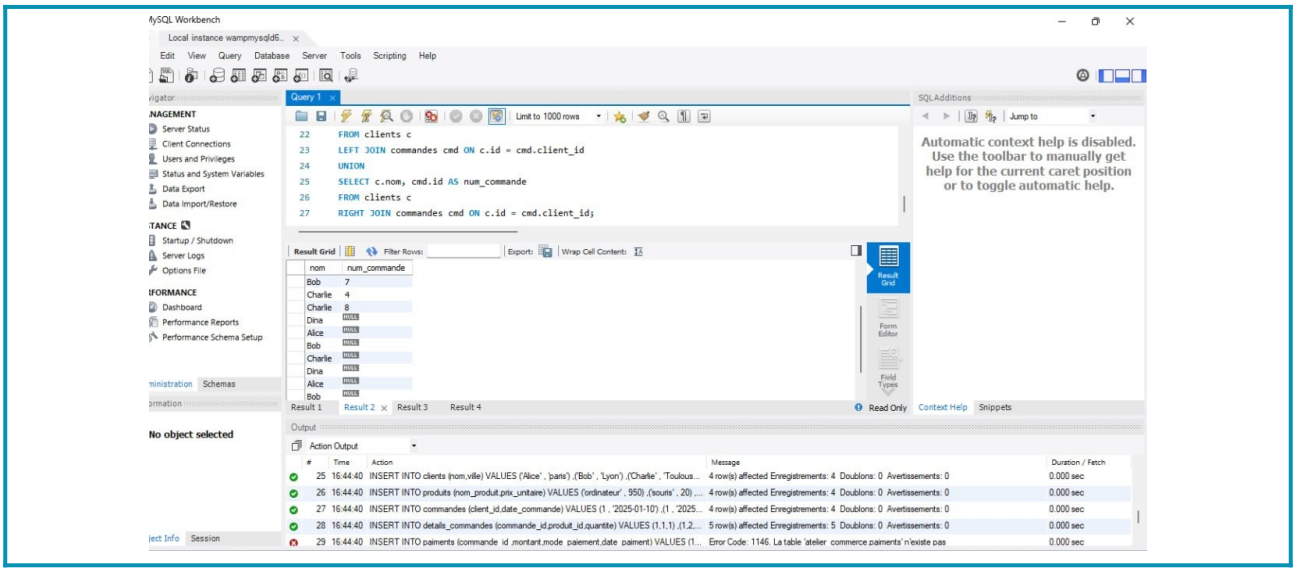


Figure 2.2 — LEFT JOIN : tous les clients, y compris ceux sans commande (num_commande = NULL, par exemple Dina).

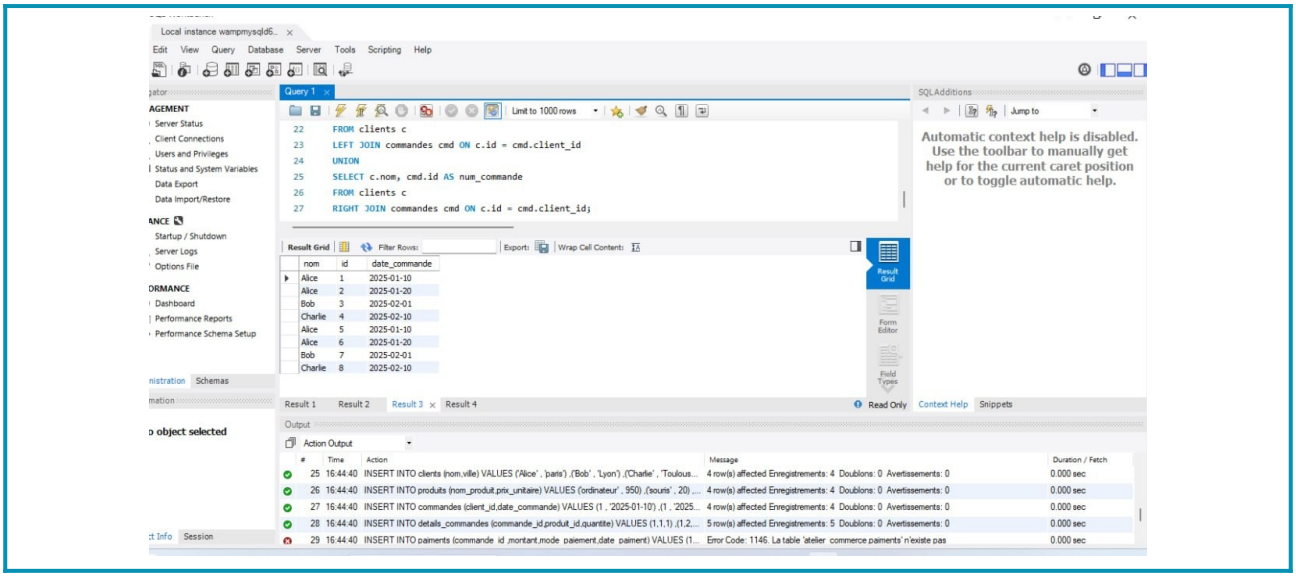


Figure 2.3 — RIGHT JOIN : toutes les commandes avec le nom du client et la date de commande.

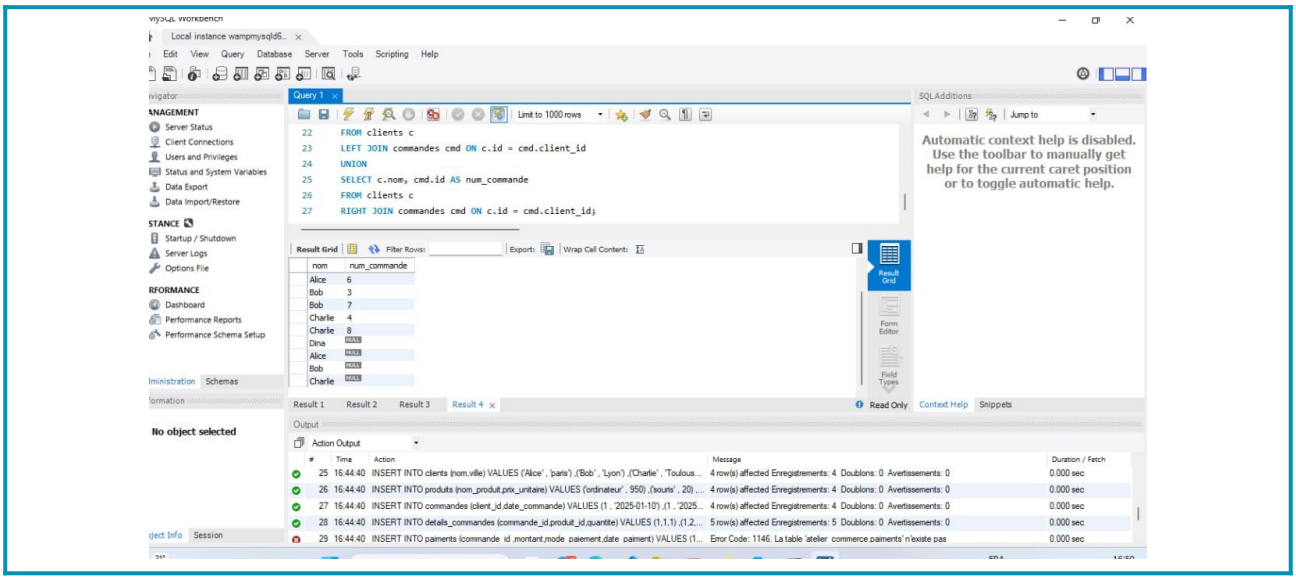


Figure 2.4 — FULL JOIN simulé avec UNION : résultat combiné des deux jointures.

3 Agrégations et calculs

Objectif Calculs avec SUM, AVG, GROUP BY et HAVING : total par commande, total dépensé par client (supérieur à 500) et prix moyen par produit.

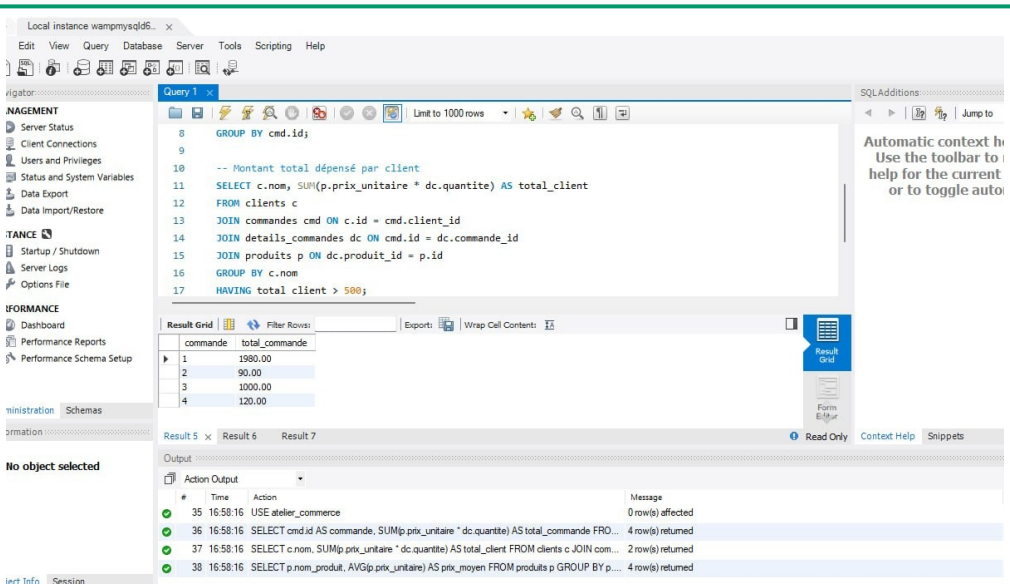


*Figure 3.1 — Montant total par commande : SUM(prix_unitaire * quantite) groupé par commande.*

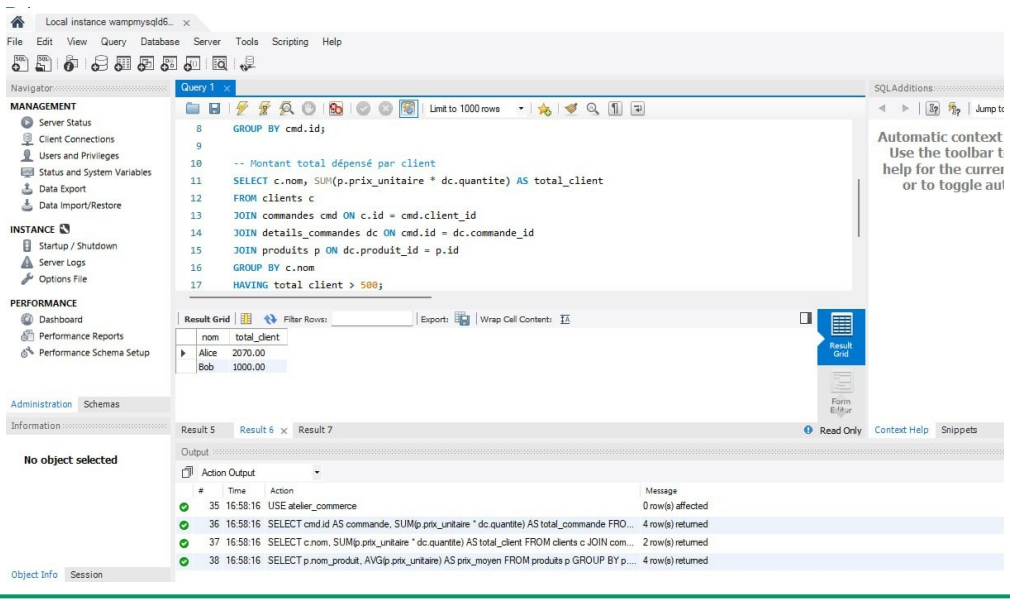


Figure 3.2 — Montant total dépensé par client avec HAVING total_client > 500.

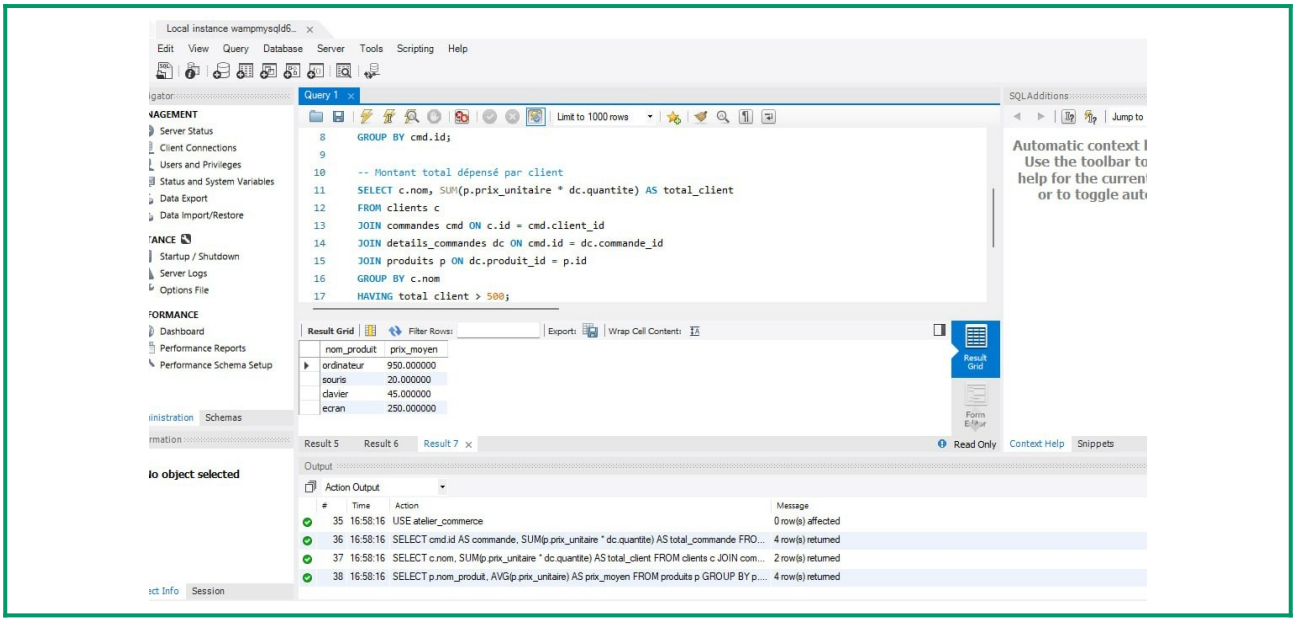


Figure 3.3 — *Prix moyen par produit avec AVG et GROUP BY.*

4 Fonction SQL

Objectif Utilisation de fonctions SQL : classification des clients avec IF (Premium / Standard), calcul du délai de paiement avec DATEDIFF, et classement des meilleurs clients avec ORDER BY et LIMIT.

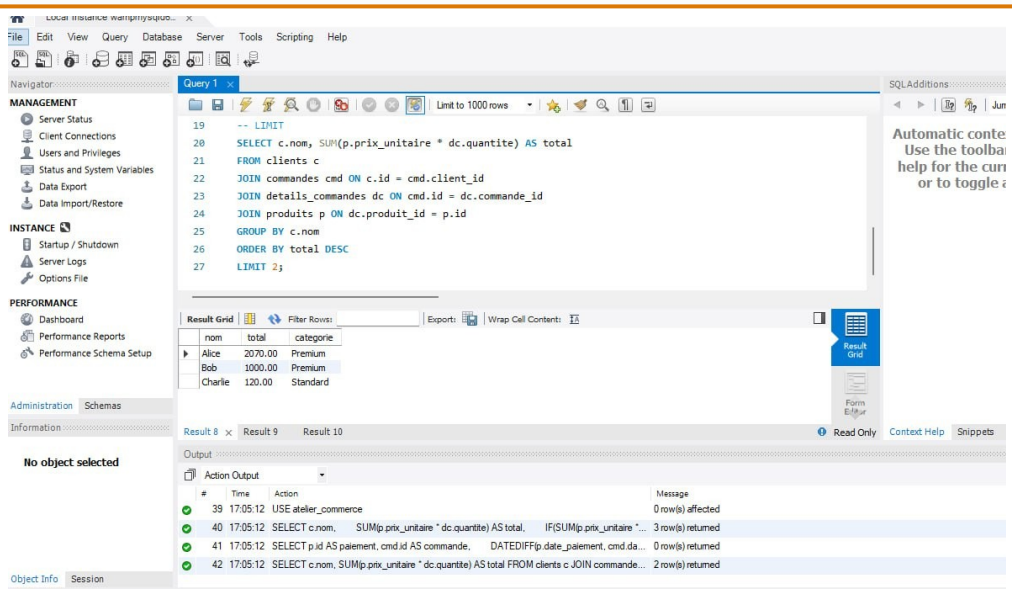


Figure 4.1 — Catégorie du client (Premium ou Standard) selon le total de ses achats avec la fonction IF.

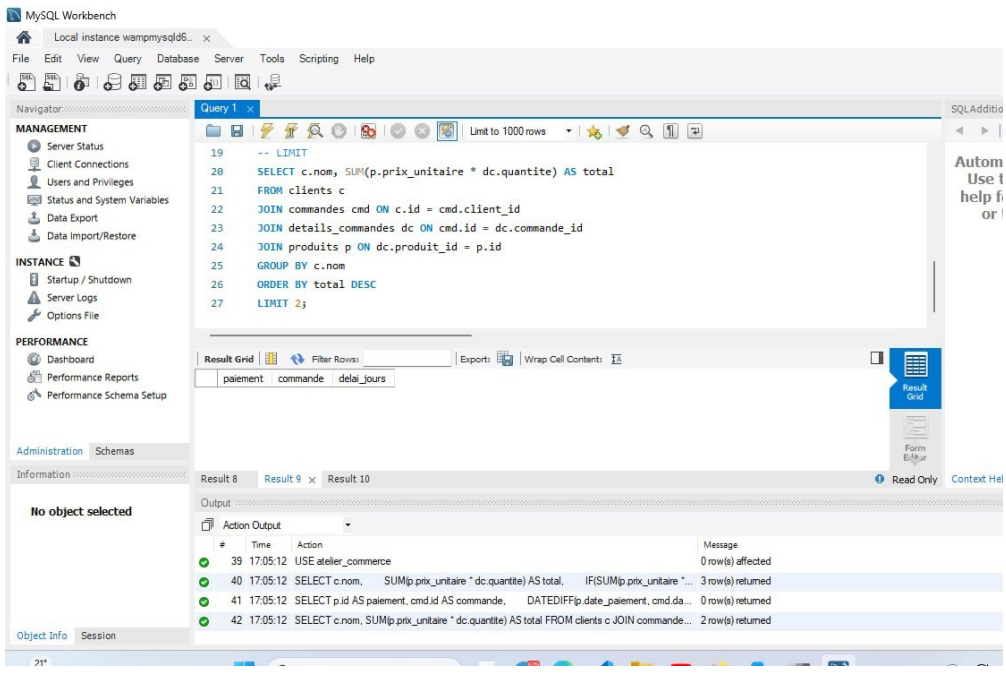


Figure 4.2 — Délai entre la commande et le paiement avec DATEDIFF (aucune ligne car la table paiements était vide).

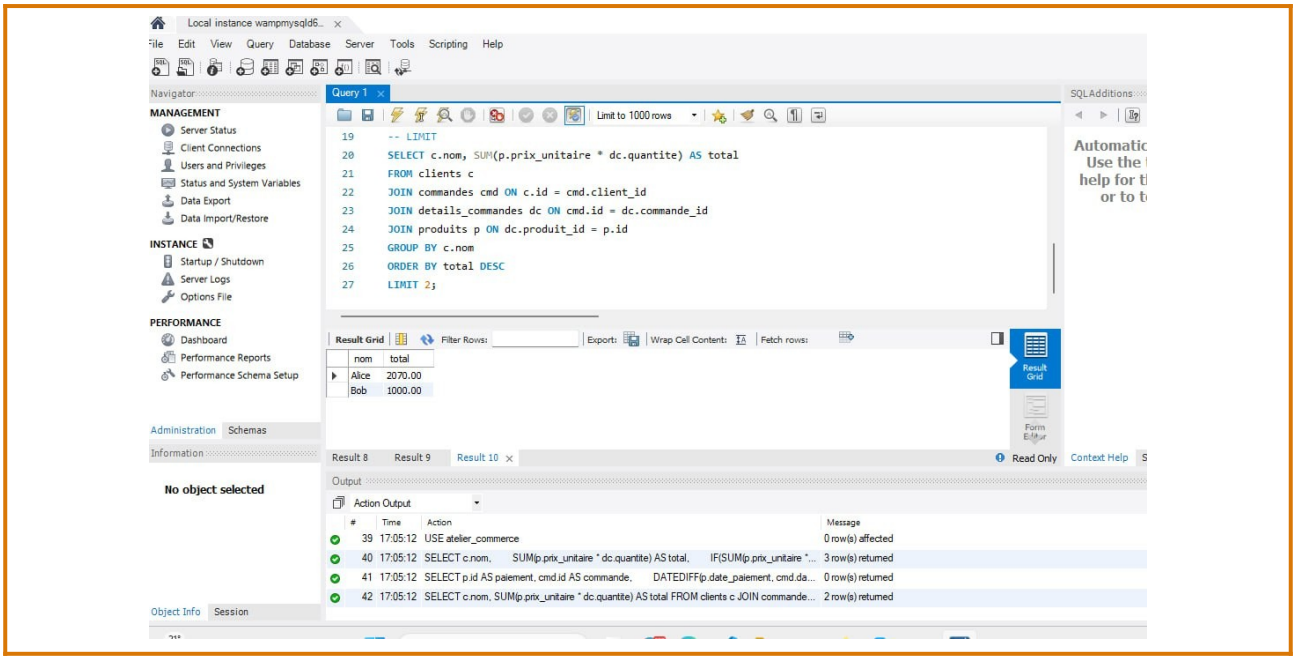


Figure 4.3 — Les deux meilleurs clients avec ORDER BY total DESC et LIMIT 2.

5 IN / NOT IN / EXISTS

Objectif Sous-requêtes pour trouver les clients ayant (ou non) effectué un paiement, avec IN, NOT IN et EXISTS.

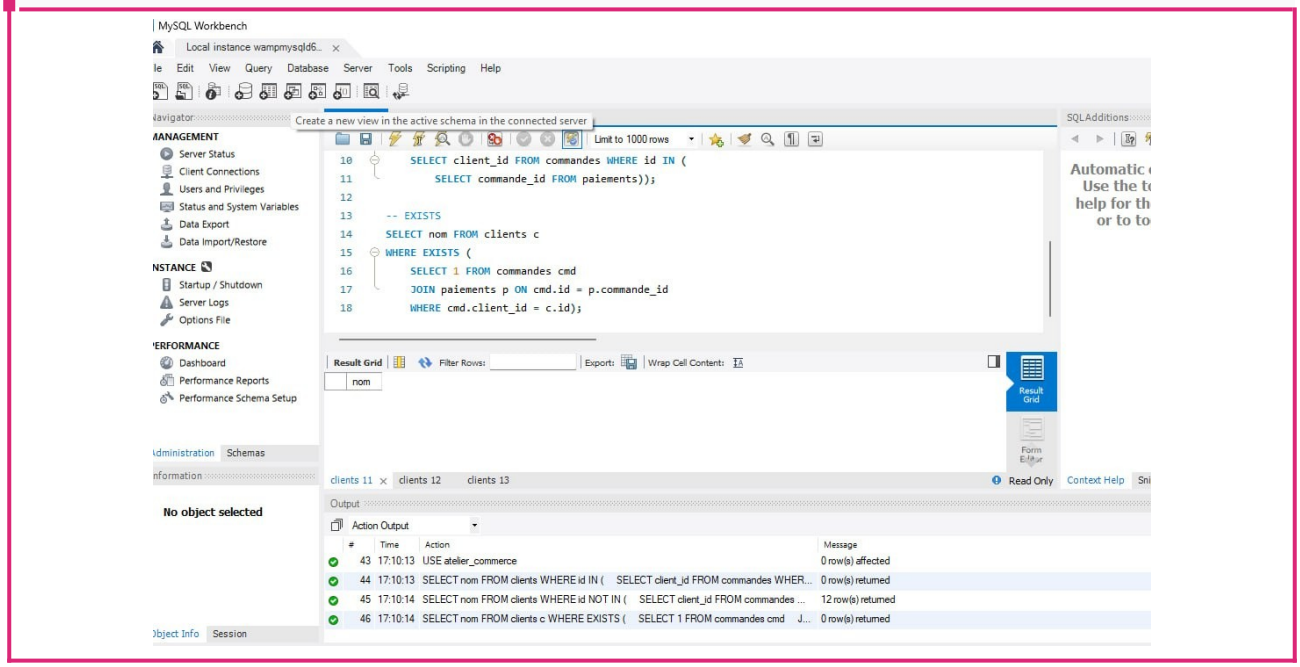


Figure 5.1 — *IN : clients ayant une commande payée (0 ligne : aucun paiement enregistré).*

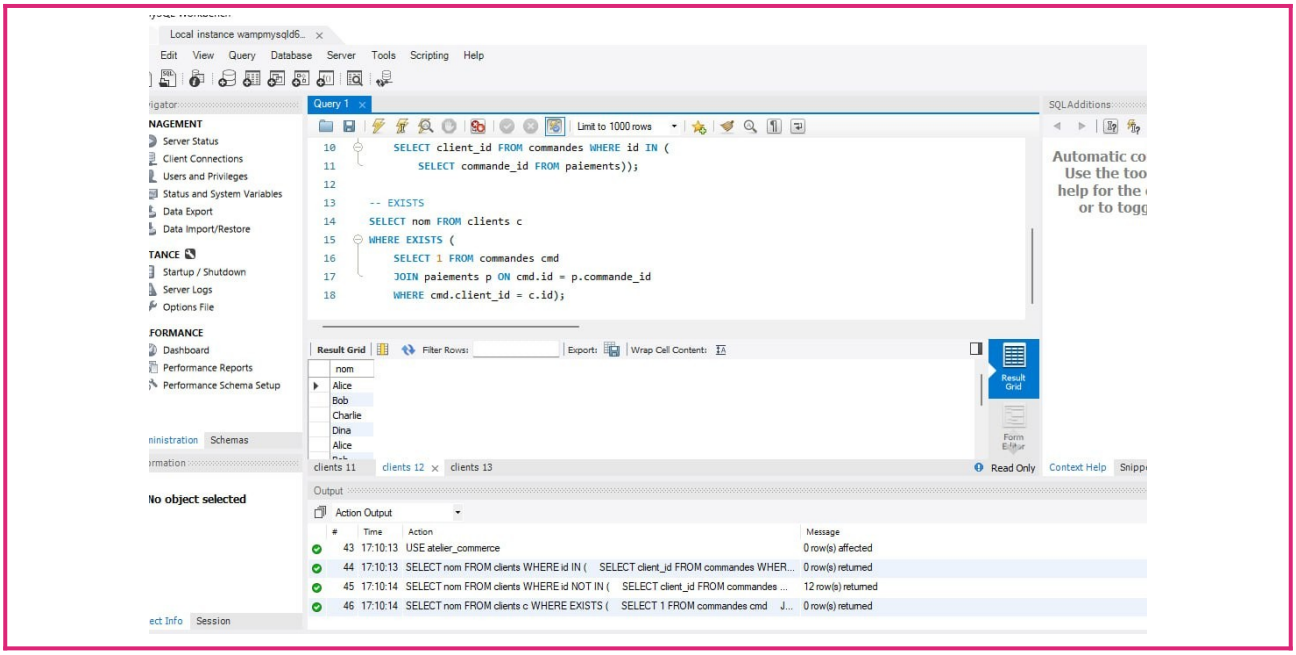


Figure 5.2 — *NOT IN : clients sans commande payée (tous les clients apparaissent).*

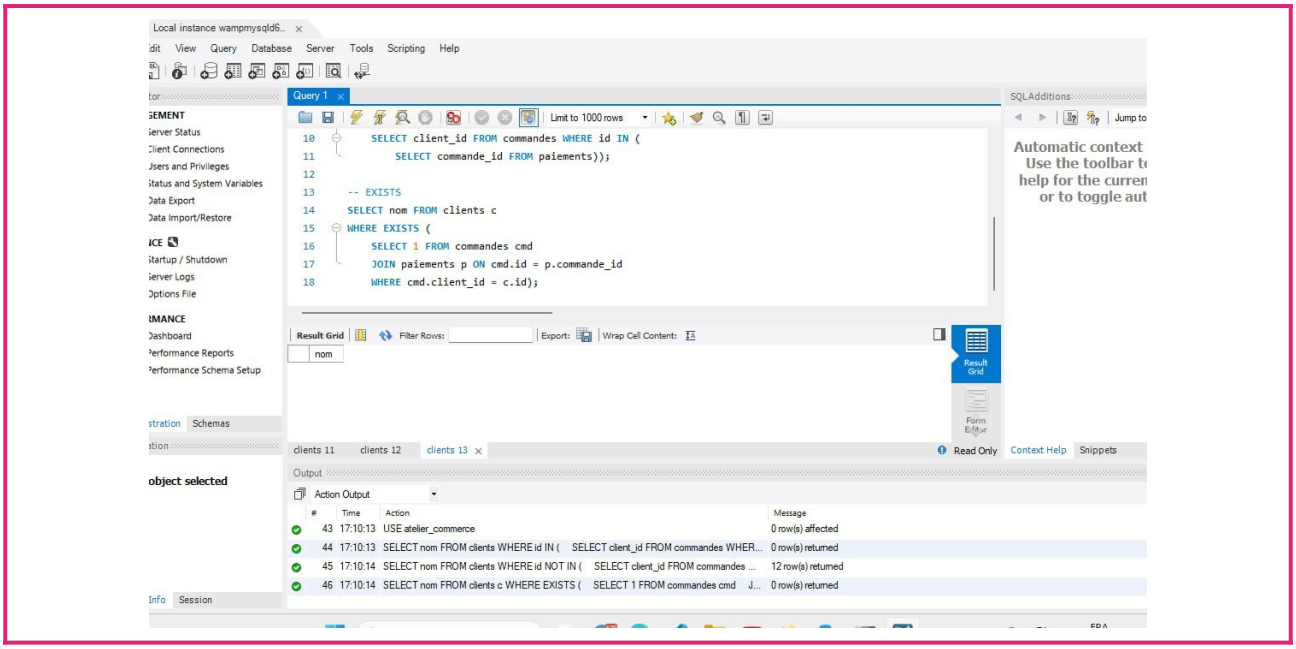


Figure 5.3 — *EXISTS : même logique avec une sous-requête corrélée (0 ligne).*

6 Optimisation : Index et Vues

Objectif Amélioration des performances avec un index sur la colonne nom_produit (vérifié avec EXPLAIN) et création de la vue vue_ventes qui regroupe les informations de vente.

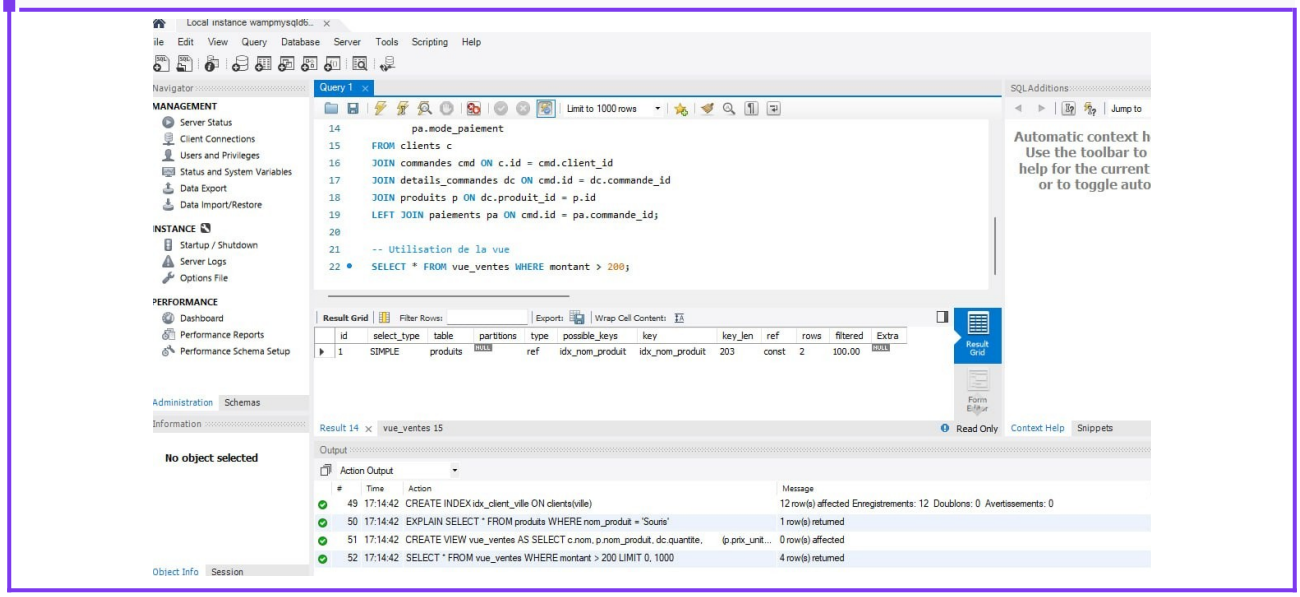


Figure 6.1 — EXPLAIN : la requête utilise l'index idx_nom_produit (type ref, 2 lignes examinées).

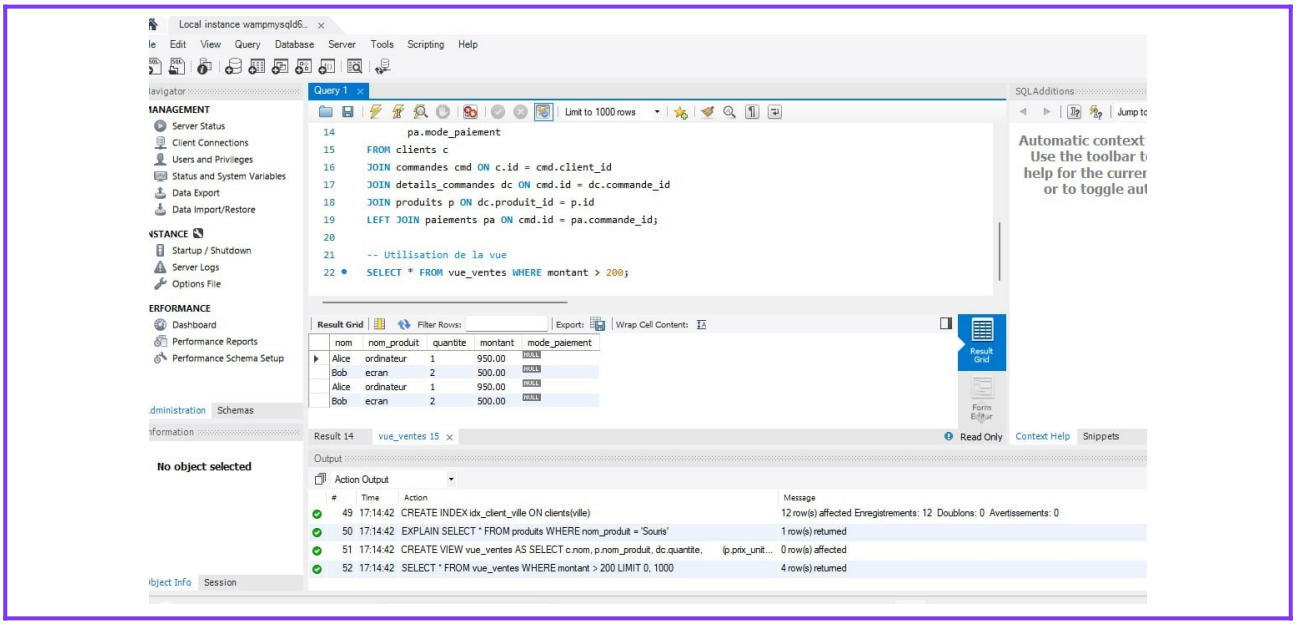


*Figure 6.2 — Création de la vue vue_ventes puis utilisation : SELECT * FROM vue_ventes WHERE montant > 200.*

7 DCL - Gestion des utilisateurs

Objectif Gestion des droits : GRANT pour les utilisateurs vendeur et manager, FLUSH PRIVILEGES, vérification avec SHOW GRANTS et retrait du droit UPDATE avec REVOKE.

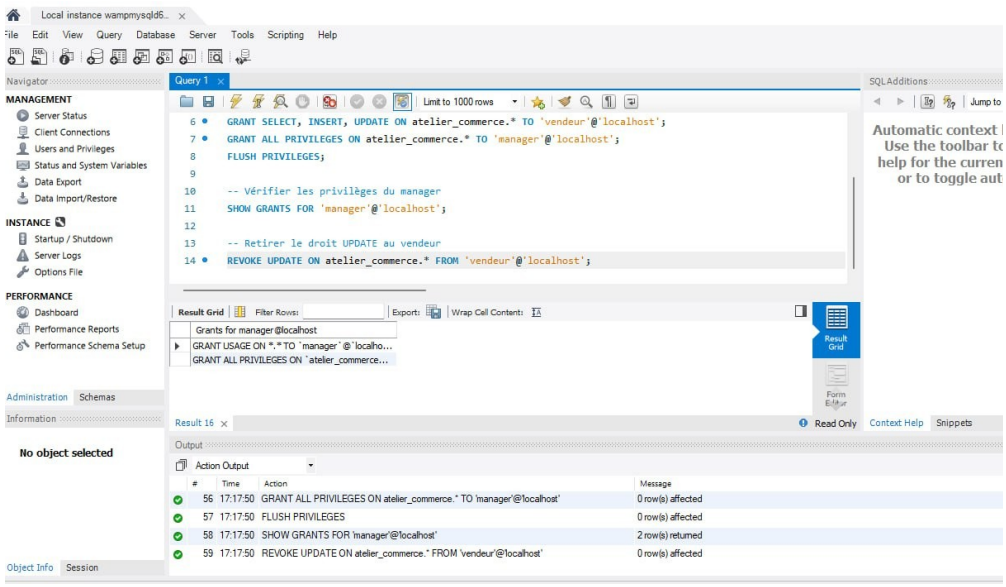


Figure 7.1 — *Attribution des privilèges (GRANT), vérification des droits du manager (SHOW GRANTS) et REVOKE UPDATE pour le vendeur.*